EBU5204 Operating Systems (2023/24)

LAB 2: Process monitoring in Linux

Note: This lab sheet is important as you will have to attempt a lab test on QMPlus based on the commands provided in this lab sheet. The lab test will be assessed and will contribute to the final marks.

Aim

The aim of this lab is to find out how you can discover information about processes in Linux systems (state, memory requirements and usage, running time, etc.). The main commands we will use in this lab are:

- `ps` (process status), `pgrep`, `pkill`, `kill`, `killall`
- `top` (an interactive, real-time view of processes running on a system, together with all sorts of useful information about the processes).
- `pstree` (process information in tree form showing parent/child relationships)

You should be able to:

1. Open a terminal in Linux and use commands (e.g., those from **Lab 1**) to navigate in your directories and examine your files and their properties (e.g., using `r-w-x` permissions).
2. Read the man pages of commands when you need to find out information about specific options e.g., `man ps` and find out about commands in https://linuxcommand.org/.

This lab sheet is not meant to act as a stand-alone tutorial. It gives you some basic information about the commands, but it your responsibility to explore further how you can get the commands to work for you (e.g., using man pages, exploring https://linuxcommand.org/ or simply playing with the commands in a Linux terminal).

Note also that output formats may vary slightly between machines and shells and will most likely vary if you try these commands on a Mac or certain distributions of **Ubuntu.**

1 The `ps` (process status) command

The `ps` command (also known as "process status") gives various types of information about processes running. Depending on what options you supply to the command, you can select different processes to view, and you can display different types of information for these processes. It supports searching for processes by user, group, process id or executable name.

a. List all processes in current shell (terminal window)

If you simply run `ps` without any other arguments, you will get a list of processes for the current running shell (i.e., from the terminal you ran the command from), something like given in Figure 1:

```
flakhani@ubuntu22:~$ ps
    PID TTY          TIME CMD
   2001 pts/0    00:00:00 bash
   2038 pts/0    00:00:00 ps
flakhani@ubuntu22:~$
```

Figure 1

where PID is the unique process id, TTY the console (terminal) that the user is logged into, TIME the amount of time the process has been running and CMD the name of the command that launched the process.

b. Display all processes running on a system

To list all processes for a system, use the `-e` option (typically the output will be very long so may want to pipe to a paginator such as `more`): See Figure 2

```
flakhani@ubuntu22:~$ ps -e | more
    PID TTY          TIME CMD
      1 ?        00:00:01 systemd
      2 ?        00:00:00 kthreadd
      3 ?        00:00:00 rcu_gp
      4 ?        00:00:00 rcu_par_gp
      5 ?        00:00:00 slub_flushwq
      6 ?        00:00:00 netns
      7 ?        00:00:00 kworker/0:0-events
      8 ?        00:00:00 kworker/0:0H-events_highpri
      9 ?        00:00:00 kworker/u8:0-events_unbound
     10 ?        00:00:00 mm_percpu_wq
     11 ?        00:00:00 rcu_tasks_kthread
     12 ?        00:00:00 rcu_tasks_rude_kthread
     13 ?        00:00:00 rcu_tasks_trace_kthread
     14 ?        00:00:00 ksoftirqd/0
     15 ?        00:00:00 rcu_preempt
```

Figure 2

Note that you get the same output with issuing the command: `ps -A`.

A useful hint: You can **redirect** the output of any command in Linux so instead of appearing on the screen, it can be saved on to a file; for example:

`ps -e > ps_output.txt` will save the output of the command in the text file `ps_output.txt`. See Figure 3. You can open and examine the file by using the `cat` command.

```
flakhani@ubuntu22:~$ ps -e >ps_output.txt
flakhani@ubuntu22:~$ ls
classos.py  Documents  EBU5204     favourite2  hello3.py  link     Pictures       Public
Desktop     Downloads  favourite   hello2.py   Hello.py   Music    ps_output.txt  simplefork.py
flakhani@ubuntu22:~$ c
```

Figure 3

Check **http://linuxcommand.org/lc3_lts0070.php** for redirection in Linux, it is very useful to know.

If the `TTY` value is set to "?" this means that the process is not associated with any user terminal (e.g., it can be a system process).

More useful information can be given if you combine this with the `-f` (or `-F`) option:

For example, see Figure 4 the output from `ps -ef` command:



Figure 4

Here we see the addition of another useful piece of information for a process: in the `PPID` column we see the **id of the parent process**. For example, the parent process id of the command I ran in this example "`ps -ef | more`" is 2001 and it corresponds to the terminal window (shell) that I ran the command from (make sure you can see this in the example above in Figure 4) - `bash`.

You can combine any of these commands with the `grep` command using pipes – `grep` looks for a pattern in a text file or a text stream and returns it as its output. For example: `ps -ef | grep bash` in the example above will return the line of the output that contains the strong 'bash'. See Figure 5.

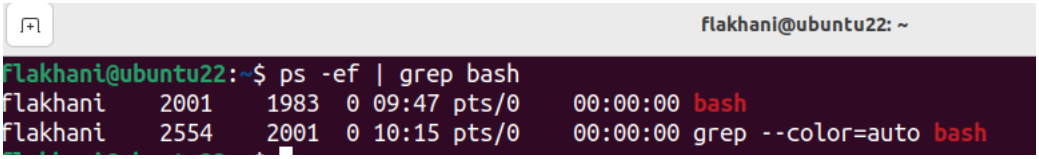


Figure 5

This is a very useful combination and one you should get used to. `grep` is a very powerful utility in Linux.

An alternative way to use the `ps` command using the BSD syntax (Berkely Standard Distribution) is without "-" before the various options. A very popular format is `ps aux`: See Figure 6.

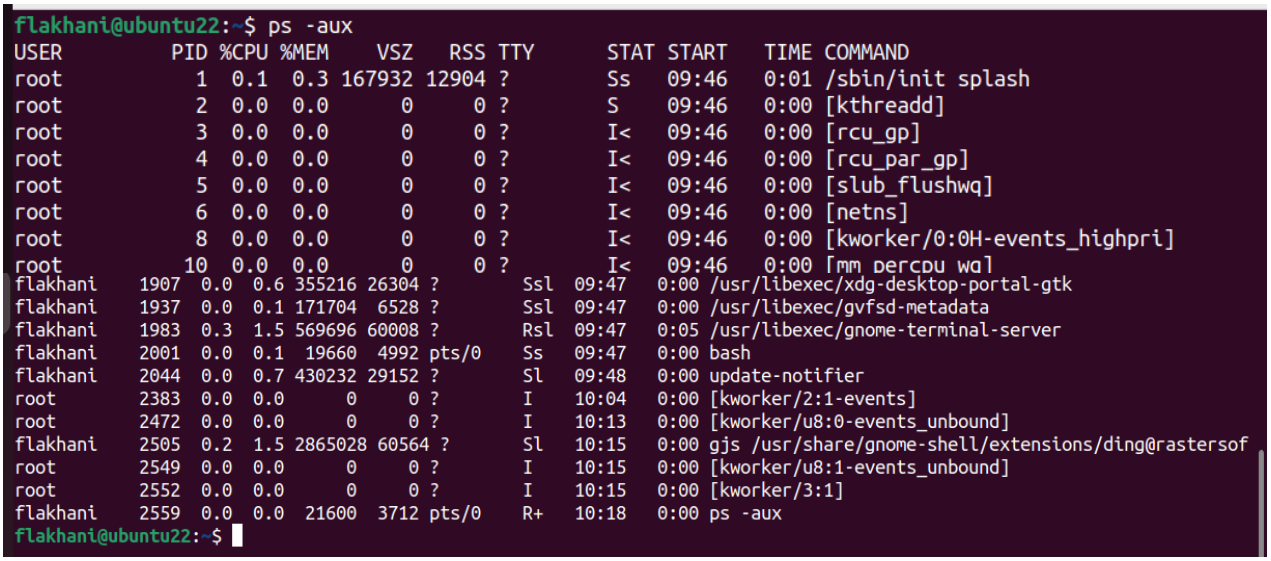


Figure 6

The **`STAT`** column gives information about the **state** in which each process in the list currently is.

Other common and useful combinations of options are: `ps -def` and `ps -elf` (it is recommended that you explore them and try to make sense of the information in the columns).

c. Display processes running for a specific user

We can combine the previous command with the `-u <user_id>` option and display information for processes running by a specific user, as in the Figure 7

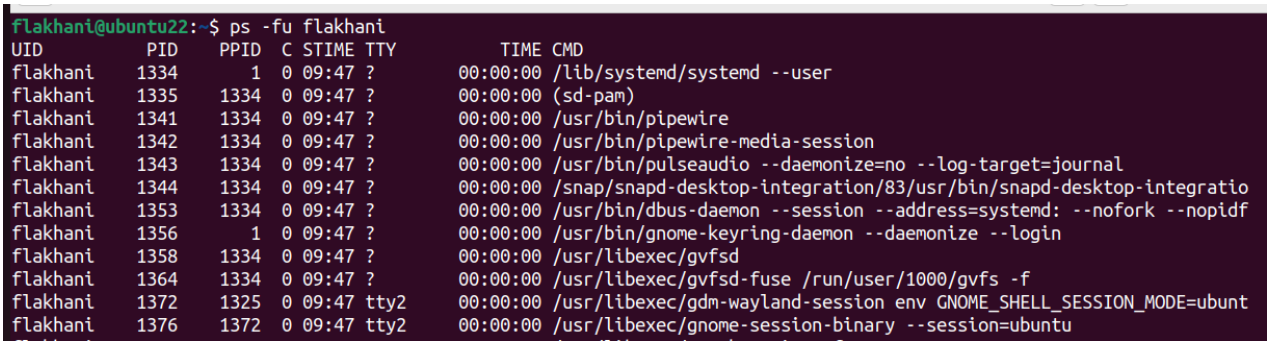


Figure 7

You can get a longer listing with details about processes with the `-l` option, see Figure 8

```
flakhani@ubuntu22:~$ ps -lfu flakhani
F S UID          PID    PPID  C PRI  NI ADDR SZ WCHAN  STIME TTY          TIME CMD
4 S flakhani    1334       1  0  80   0 -  5152 ep_pol 09:47 ?        00:00:00 /lib/systemd/systemd --user
5 S flakhani    1335    1334  0  80   0 - 42510 -      09:47 ?        00:00:00 (sd-pam)
0 S flakhani    1341    1334  0  69 -11 - 12058 ep_pol 09:47 ?        00:00:00 /usr/bin/pipewire
0 S flakhani    1342    1334  0  80   0 -  8027 ep_pol 09:47 ?        00:00:00 /usr/bin/pipewire-media-session
0 S flakhani    1343    1334  0  69 -11 - 226624 do_pol 09:47 ?       00:00:00 /usr/bin/pulseaudio --daemonize=no --log
4 S flakhani    1344    1334  0  80   0 - 19063 do_wai 09:47 ?        00:00:00 /snap/snapd-desktop-integration/83/usr/b
0 S flakhani    1353    1334  0  80   0 -  4119 ep_pol 09:47 ?        00:00:00 /usr/bin/dbus-daemon --session --address
1 S flakhani    1356       1  0  80   0 - 62404 do_pol 09:47 ?        00:00:00 /usr/bin/gnome-keyring-daemon --daemoniz
0 S flakhani    1358    1334  0  80   0 - 62326 do_pol 09:47 ?        00:00:00 /usr/libexec/gvfsd
0 S flakhani    1364    1334  0  80   0 - 95839 futex_ 09:47 ?        00:00:00 /usr/libexec/gvfsd-fuse /run/user/1000/g
4 S flakhani    1372    1325  0  80   0 - 42770 do_pol 09:47 tty2     00:00:00 /usr/libexec/gdm-wayland-session env GNO
0 S flakhani    1376    1372  0  80   0 - 57988 do_pol 09:47 tty2     00:00:00 /usr/libexec/gnome-session-binary --sess
0 S flakhani    1395    1334  0  99   - - 161068 do_pol 09:47 ?       00:00:01 /usr/libexec/tracker-miner-fs-3
0 S flakhani    1420    1334  0  80   0 - 136393 do_pol 09:47 ?       00:00:00 /usr/libexec/xdg-document-portal
0 S flakhani    1423    1334  0  80   0 - 61203 do_pol 09:47 ?        00:00:00 /usr/libexec/xdg-permission-store
0 S flakhani    1459    1334  0  80   0 - 25140 do_pol 09:47 ?        00:00:00 /usr/libexec/gnome-session-ctl --monitor
0 S flakhani    1471    1334  0  80   0 - 150451 do_pol 09:47 ?       00:00:00 /usr/libexec/gnome-session-binary --syst
0 S flakhani    1488    1471  0  80   0 - 78219 do_pol 09:47 ?        00:00:00 /usr/libexec/at-spi-bus-launcher --launc
```

Figure 8

Some interesting fields are: **PRI** (the priority of the process for scheduling), **NI** (the 'niceness' of the process, also linked with priority - the higher the value, the lower the priority. The default nice value is 0 on Linux systems.). **SZ** displays the size of the process in memory. The value of the field is the number of pages the process is occupying. On Linux systems, a page is typically 4,096 bytes.

There are numerous other options, some allowing to filter processes by user group, pid, etc., some others allowing to display the process hierarchy in a tree style, some giving more specific information about memory usage, etc.

2 pgrep

The `pgrep` command is based on a very popular utility, `grep`, which searches for a pattern in e.g., a text file, and then outputs any matches to that pattern. `pgrep` allows users to look up process information based on e.g., name, owner and other information.

For example, to find **the pid of a specific process** (e.g., `bash` of the shell I am using in as many terminal windows as I have open) owned by me: `pgrep bash -u flakhani`

```
flakhani@ubuntu22:~$ pgrep bash -u flakhani
2001
flakhani@ubuntu22:~$
```

Figure 9

If you want to use the command to print out **the number of processes owned by a specific user**, you can do it by: `pgrep -u flakhani -c`

```
flakhani@ubuntu22:~$ pgrep -u flakhani -c
68
flakhani@ubuntu22:~$
```

Figure 10

To get the complete path to the command that corresponds to the matching process, you can add the `-a` option: `pgrep -u flakhani -a` will output a long list of processes, like:

`2160 /usr/bin/firefox`

`2167 /usr/lib/gvfs/gvfsd` etc.

More options can be found in the man pages and online.

3 Killing processes: kill and pkill

In Linux you can terminate a process by sending a specific **signal** to it. The 'act' is called killing a process and commands that make this happen include `kill` and `pkill`.

There are of course certain **rules** when it comes down to killing processes: a user can kill any of the processes owned by him/her, but not those owned by other users. A user can also not kill any system processes. The **root** user can kill pretty much anything ☺.

To kill a process with the `kill` command, we must know its **pid** (we saw ways of getting this information in previous sections). For example, `kill 1234` will kill the process with pid 1234.

We can use the **`pkill`** command to kill processes by name e.g., `pkill firefox`.

We can use the **`killall`** command to kill all instances of a process, by name. For example, `killall firefox` will kill all instances of the `firefox` process running at that time and owned by me.

4. Other commands worth knowing about

`pstree`

The `pstree` command is similar to `ps` in that it can be used to show all of the processes on the system along with their PIDs. However, it differs in that it presents its output in a *tree diagram* that shows how processes are related to each other and in that it provides less detailed information about each process than `ps`. As you can imagine, the whole hierarchy tree of all the processes running on a system can be very long so you can search for specific users of processes / programs with command line options (for example, `pstree -U flakhani` would display the part of the tree that has processes owned by user `flakhani`).

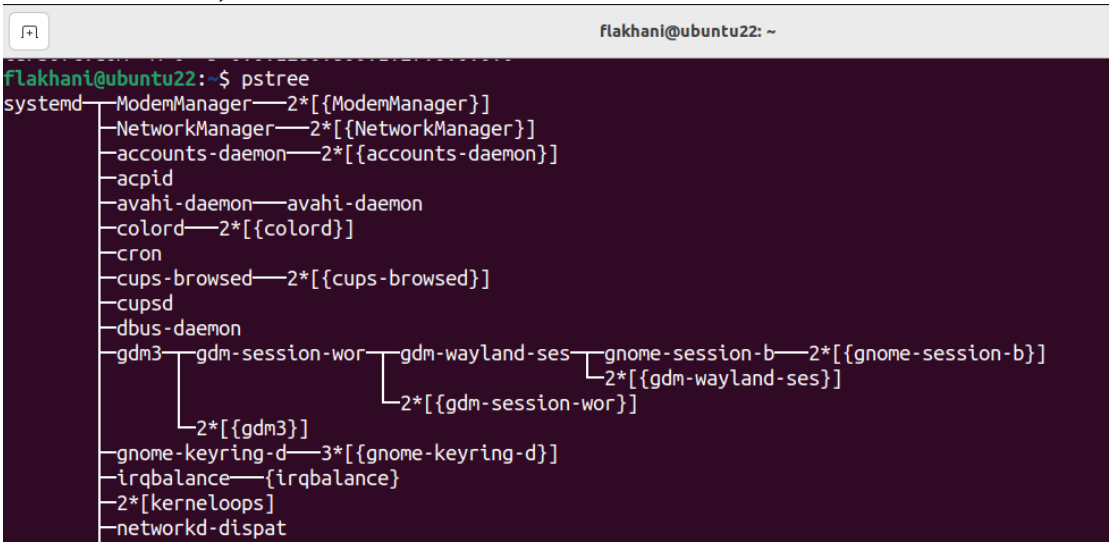


Figure 11

top

`top` is an interactive tool that allows monitoring the processes running on a system. It displays one line per process with various columns that contain CPU related, memory related, or more general information and it is more similar to interactive GUI-based tools for process monitoring (you can for example choose which field to sort processes by). A very good guide is available at: https://www.booleanworld.com/guide-linux-top-command/, or https://www.geeksforgeeks.org/top-command-in-linux-with-examples/.

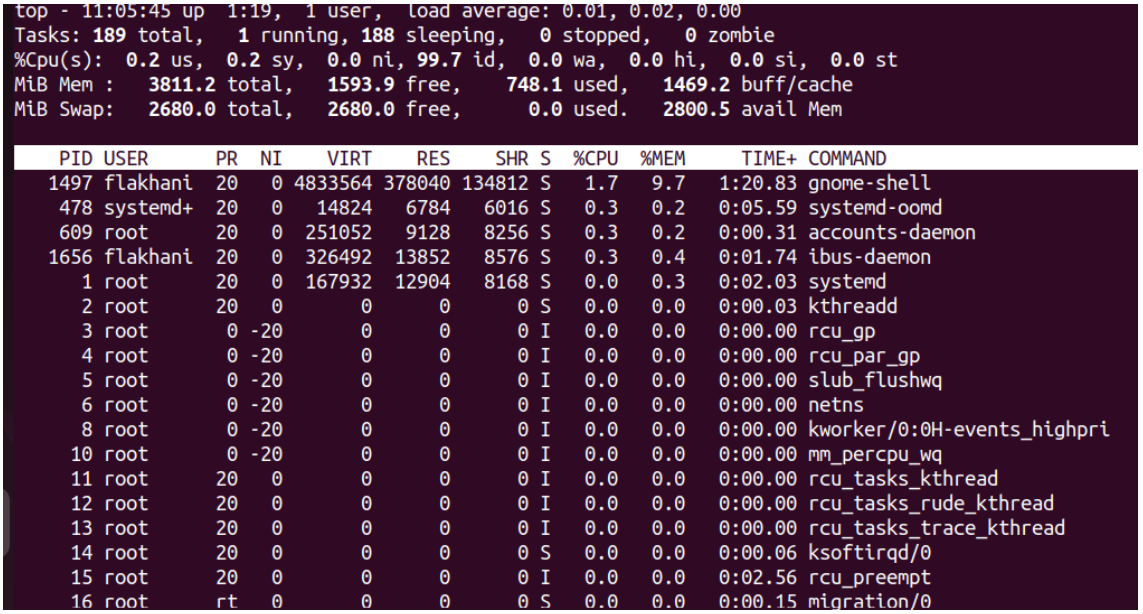


Figure 12

In terms of the fields of information displayed for each process, we have seen some of them previously. Some others worth noting (but more related to memory management so just keep them in mind for later when learning new topics in Teaching Blocks 3 and 4):

- **VIRT** – the amount of virtual memory used by a process: code, data and shared libraries plus pages that have been swapped out
- **RES** – the resident part of a process: how much of it resides in the physical memory.
- **SHR** – shows you the size of potentially shared memory segments for a process
- **S**: Process state

Practice tasks:

Experiment with the commands given in the sheet

- Run the examples from a Linux terminal, try variations and experiment with different options. Keep in mind that if you use a Mac or certain versions of Ubuntu, output may vary from the examples given.
- If some of the output or command options are not clear:
    - Look for help through the `man` pages.
    - Ask for help during your lab time.